Cheat Sheet Presentasi

Baca ini sebelum maju. Isinya jawaban untuk pertanyaan yang paling mungkin dikeluarkan dosen.

BAGIAN 1 — Kalimat Pembuka (hafalkan ini)

> "Kami membuat sistem yang memprediksi **probabilitas nasabah gagal bayar**.
> Masukannya data finansial nasabah, keluarannya satu angka persen.
> Modelnya neural network yang dilatih pada 30.000 data historis dari UCI,
> dan diuji pada 6.000 data yang belum pernah dilihat, dengan hasil AUC 0,773."

Selesai. Sisanya jawaban atas pertanyaan.

BAGIAN 2 — Tanya Jawab

"Apa itu deep learning? Bedanya sama machine learning biasa?"

Machine learning itu payungnya. Deep learning adalah machine learning yang pakai **neural network berlapis banyak**.

Analogi: mau kenali wajah. Cara biasa, lo tulis aturannya manual — "kalau ada dua mata, satu hidung, satu mulut, berarti wajah". Ribet dan gampang salah. Deep learning kasih 10.000 foto wajah, biar dia nyari sendiri aturannya.

Yang penting dijawab: "Deep" artinya banyak lapisan. Punya kami 2 lapisan tersembunyi.

"Jelaskan cara kerja neural network-nya"

Bayangkan mesin dengan banyak **tombol pengatur (bobot)**. Data masuk dari kiri, lewat tombol-tombol itu, keluar angka di kanan.

Awalnya tombolnya diatur acak, jadi keluarannya ngawur. Lalu kita bilang: "harusnya 1, kamu bilang 0,3 — salahnya 0,7". Sistem otomatis memutar semua tombol sedikit ke arah yang bikin kesalahan mengecil. Ulangi 30 kali dengan 24.000 data.

Selesai latihan, tombolnya sudah pas. Itu yang disebut model terlatih.

Istilah teknisnya: forward pass → hitung loss → backpropagation → update bobot. Tapi jangan mulai dari istilah ini, mulai dari analogi tombol.

"Kenapa outputnya probabilitas, bukan langsung layak/tidak layak?"

Karena **keputusan kredit bukan hitam-putih**. Dua nasabah dengan PD 21% dan 22% secara statistik hampir sama, tapi kalau dipaksa jadi "layak/tidak layak" mereka bisa jatuh ke sisi berbeda hanya karena pembulatan.

Dengan probabilitas:
- Informasinya lebih kaya (21% vs 80% beda jauh)
- Bisa dihitung **expected loss** = PD × kerugian
- Ambang keputusan bisa diubah sesuai kebijakan tanpa melatih ulang model

Ini inti topik Kelompok 2: **probabilitas sebagai dasar pengambilan keputusan**.

"Dari mana angka 18,8% itu? Kok bisa percaya?"

Angka itu keluaran sigmoid dari model. Latihannya diarahkan supaya angka yang keluar **cocok dengan frekuensi kejadian nyata**. Kalau model bilang "20%" untuk 1.000 nasabah, idealnya sekitar 200 yang benar-benar gagal bayar.

Buktinya: AUC 0,773 pada 6.000 data uji yang tidak pernah dilihat saat latihan. Kalau modelnya ngawur, AUC-nya 0,5.

Jujurnya: ini estimasi statistik, bukan ramalan pasti. Kami tidak bilang "nasabah ini PASTI gagal bayar", tapi "dari 100 nasabah dengan profil seperti ini, sekitar 19 diperkirakan gagal bayar".

"Apa itu AUC 0,773? Bagus atau jelek?"

AUC mengukur: kalau saya ambil satu nasabah yang benar-benar gagal bayar dan satu yang benar-benar lancar, berapa peluang model menempatkan yang gagal bayar di posisi lebih berisiko?

- AUC 0,5 = tebak acak (kaya lempar koin)
- AUC 0,773 = **77,3% benar** dalam perbandingan berpasangan
- AUC 1,0 = sempurna (tidak mungkin di dunia nyata)

Untuk credit scoring, AUC 0,70–0,80 sudah tergolong **baik**. Bank komersial biasanya 0,75–0,85 karena punya data internal yang jauh lebih lengkap.

"Kenapa pakai data Taiwan, bukan Indonesia?"

Jawaban jujur dan ini justru poin plus: data kredit per-nasabah di Indonesia bersifat konfidensial, diatur OJK. Tidak ada yang mempublikasikan "nasabah A gagal bayar" — itu rahasia bank.

Yang tersedia publik hanya statistik agregat (NPL per sektor), dan itu tidak bisa dipakai melatih model per-nasabah.

Jadi kami pakai dataset publik UCI yang diakui akademik. Sistemnya dirancang sehingga dataset internal bank bisa langsung dipakai menggantikan data Taiwan tanpa mengubah arsitektur — cukup ganti file dan latih ulang.

Jangan bilang "karena tidak ada data Indonesia" lalu berhenti. Bilang alasannya: konfidensialitas.

"Kenapa 30.000 data dibagi jadi 24.000 latihan dan 6.000 uji?"

Karena model harus diuji pada data yang **belum pernah dia lihat**. Kalau diuji pakai data latihan, hasilnya pasti bagus tapi bohong — seperti kasih soal ujian yang jawabannya sudah dihafal.

24.000 untuk belajar, 6.000 untuk ujian. Ini praktik standar (80/20).

"Apa itu overfitting?"

Model yang **menghafal** data latihan, bukan memahami pola. Hasil di data latihan bagus, tapi di data baru jelek.

Analogi: siswa yang hafal jawaban soal tahun lalu. Dikasih soal yang sama, nilainya 100. Dikasih soal baru, nilainya 30.

Cara mencegah di sistem kami: dropout 30% (sebagian neuron dimatikan acak saat latihan, memaksa model tidak bergantung pada satu jalur saja).

"Apa itu dropout? Kenapa 0,3?"

Saat latihan, 30% neuron dimatikan secara acak di setiap langkah.

Tujuannya: mencegah model "menghafal" dengan bergantung pada neuron tertentu. Kalau neuron favoritnya dimatikan, dia harus belajar pakai yang lain.

0,3 adalah nilai umum yang bekerja baik. Kalau 0,5 terlalu banyak informasi hilang, kalau 0,1 kurang efektif mencegah overfitting.

"Kenapa pakai sigmoid di akhir?"

Sigmoid memaksa output berada di rentang 0–1, sehingga bisa dibaca sebagai probabilitas.

Rumusnya: `σ(x) = 1 / (1 + e^(-x))`

Kalau x sangat besar → hasilnya mendekati 1. Kalau x sangat negatif → mendekati 0. Kalau x = 0 → hasilnya tepat 0,5.

Tanpa sigmoid, output bisa angka apa saja (-500, 2000) dan tidak bisa disebut probabilitas.

"Apa itu loss function? Kenapa pakai BCE?"

Loss = angka yang mengukur seberapa salah model. Latihan = mengecilkan angka ini.

Kami pakai **Binary Cross-Entropy**, rumusnya:

```
L = −[y·log(p) + (1−y)·log(1−p)]
```

Kenapa bukan sekadar "selisih biasa"? Karena BCE **menghukum berat** prediksi yang sangat percaya diri tapi salah. Model bilang "pasti lancar (p=0,01)" padahal nasabah gagal bayar → hukumannya sangat besar. Ini memaksa model berhati-hati.

"Apa itu class imbalance? Kalian tangani bagaimana?"

Data tidak seimbang: 22% gagal bayar, 78% lancar. Kalau model asal jawab "semua lancar", akurasinya sudah 78% — tapi model itu **tidak berguna** karena tidak pernah mendeteksi satu pun nasabah berisiko.

Penanganan: kami beri bobot lebih besar pada kelas minoritas. Kesalahan pada nasabah gagal bayar dihitung ~3,5× lebih berat daripada kesalahan pada nasabah lancar. Ini memaksa model memperhatikan kelas minoritas.

"Apa itu embedding? Kenapa tidak one-hot?"

Fitur kategorikal (misal pendidikan: S1/SMA/SMP) tidak punya urutan angka yang bermakna. Kalau diubah jadi 1/2/3, model bisa salah kira "SMP + SMA = 2×S1".

One-hot: S1 = [1,0,0], SMA = [0,1,0], SMP = [0,0,1]. Bekerja, tapi boros dan menganggap semua kategori sama jauhnya.

Embedding: tiap kategori diwakili vektor kecil yang **dipelajari** model.
Model bisa menemukan sendiri bahwa S1 dan SMA lebih mirip dibanding S1 dan SMP.
Ini teknik yang sama dipakai di NLP modern.

"Berapa parameter modelnya? Kenapa arsitekturnya kecil?"

Sekitar 4.261 parameter. Kecil memang, dan itu **disengaja**.

Dataset kami hanya 24.000 baris dengan 23 fitur. Model raksasa (jutaan parameter) justru akan overfit — menghafal, bukan belajar. Untuk data tabungan seperti ini, model kecil biasanya lebih baik.

Kalau ditanya "kenapa tidak pakai ResNet/Transformer": karena itu untuk gambar dan teks, bukan data tabular. Memakai model raksasa untuk 23 kolom angka itu seperti pakai truk untuk antar paket satu kelereng.

"Kenapa tidak pakai XGBoost? Katanya lebih bagus untuk data tabular"

Jawaban jujur: **XGBoost sering menang** untuk data tabular. Tapi tugas ini topiknya deep learning, jadi kami pakai neural network.

Yang bisa ditambahkan: "Kami sadar XGBoost sering lebih akurat di data tabular. Neural network dipilih karena fokus pembelajaran kami adalah deep learning, dan hasilnya sudah memadai (AUC 0,773)."

Jangan defensif. Akui saja, itu justru menunjukkan lo paham lanskapnya.

"Apa keterbatasan sistem ini?"

Sebutkan 3, ini menunjukkan kedewasaan analisis:

1. **Data beda konteks** — Taiwan, bukan Indonesia. Perilaku kredit berbeda antar negara.
2. **Data historis** — model belajar dari masa lalu. Kalau kondisi ekonomi berubah drastis (krisis), akurasi turun.
3. **Tidak ada variabel makro** — suku bunga, inflasi, pengangguran tidak masuk model, padahal berpengaruh besar.

"Ini sudah online? Gimana cara deploy-nya?"

Ya, sudah. Sistem berjalan di VPS Ubuntu dan dapat diakses publik melalui Cloudflare Tunnel.

Alur sederhananya:

1. Model dilatih di laptop (30 epoch, ~1 menit)
2. Bobot model diekspor ke format NumPy
3. File di-upload ke VPS
4. Flask + gunicorn menyajikan prediksi
5. Cloudflare Tunnel menghubungkan ke internet

Pertanyaan lanjutan yang mungkin muncul — "kenapa tidak pakai PyTorch di server?"

Karena VPS-nya hanya 2 GB RAM. PyTorch makan ~1 GB. Model kami cuma butuh perkalian matriks dan ReLU, jadi dihitung manual pakai NumPy — server hanya pakai ~390 MB.

Yang penting: hasilnya **identik**. Kami uji 200 input acak, selisih maksimum 0,0000001. Itu batas presisi angka desimal komputer, bukan kesalahan model.

Kalau ditanya "kenapa pakai tunnel, bukan buka port?"

VPS-nya bertipe NAT — port masuk ditutup oleh provider. Tunnel bekerja dengan koneksi keluar dari server, jadi tidak perlu port terbuka.

"Kenapa training di laptop, bukan di server?"

VPS cuma 1 core dan 2 GB RAM. Training 30.000 data di situ akan lambat dan berisiko kehabisan memori.

Selain itu ini praktik normal di industri: **pelatihan dan penyajian dipisahkan**. Model dilatih berkala (misal sebulan sekali), tapi prediksi harus tersedia 24 jam. Server tidak perlu PyTorch hanya untuk melayani prediksi.

"Kalau modelnya di-update, harus deploy ulang?"

Ya, tapi cuma upload dua file: bobot baru (`model_weights.npz`) dan parameter standardisasi (`metrics.json`), lalu restart service. Tidak perlu install ulang apa pun.

"Kalau datanya bagus, kenapa tidak 100% akurat?"

Karena **masa depan tidak bisa dipastikan**. Dua nasabah dengan profil identik bisa berakhir beda — satu kena PHK, satu dapat promosi. Itu faktor di luar data.

Model prediksi yang jujur mengakui ketidakpastian ini. Yang mengklaim akurasi 100% justru mencurigakan (pasti overfit atau data leakage).

BAGIAN 3 — Angka yang Harus Dihafal

Angka	Arti
30.000	jumlah nasabah di dataset
22,12%	tingkat gagal bayar di dataset
24.000 / 6.000	pembagian data latihan / uji
0,773	AUC-ROC model
74,8%	akurasi pada ambang 50%
23	jumlah fitur
30	epoch pelatihan
30%	dropout rate
2	jumlah lapisan tersembunyi
4.261	jumlah parameter model
390 MB	RAM terpakai di server

BAGIAN 4 — Kalau Kehabisan Kata

Kalimat penyelamat:

- "Yang kami pelajari dari proyek ini adalah bagaimana probabilitas bisa dipakai sebagai dasar keputusan, bukan sekadar klasifikasi benar-salah."
- "Model ini bukan penentu akhir, tapi alat bantu. Keputusan tetap butuh pertimbangan manusia."
- "Kalau ditanya soal akurasi sempurna: dalam prediksi risiko, ketidakpastian itu bagian dari hasilnya."

BAGIAN 5 — Yang JANGAN Dilakukan

1. **Jangan bilang "data Indonesia"** kalau ditanya. Datanya Taiwan. Bilang alasannya: data Indonesia konfidensial.
2. **Jangan mengklaim akurasi tinggi tanpa menyebut data uji.** Selalu bilang "pada 6.000 data yang belum pernah dilihat".
3. **Jangan pakai istilah yang tidak dipahami.** Kalau ditanya "apa itu backpropagation" dan lo ragu, jawab dengan analogi tombol tadi, bukan dengan istilah yang lebih rumit.
4. **Jangan defensif kalau ditanya kelemahan.** Akui, sebutkan 3 keterbatasan dari Bagian 2. Itu menunjukkan pemahaman.